

Running DeepSeek-V4-Flash and Kimi-K3 on Consumer Hardware with SSD Expert Pack

SGLang brings the core idea of SSD-LLaMA to MoE inference: keep routed experts that do not fit in VRAM and host RAM on an NVMe SSD, load only the experts selected by the router, and use Expert Pack layout, direct I/O, pinned staging, asynchronous H2D transfers, and a GPU cache to turn SSD capacity into a practical backing tier.

1. Introduction: turning a VRAM problem into a storage problem

The total parameter capacity of DeepSeek-V4-Flash and Kimi-K3 is far beyond the VRAM of a single consumer GPU. A conventional deployment therefore needs multiple GPUs or hundreds of gigabytes, and sometimes terabytes, of host memory. That capacity requirement creates a large barrier between frontier model capability and local hardware.

SGLang's SSD-backed Expert Pack path takes a different approach. Routed expert weights remain on an NVMe SSD. The router activates only a small subset of experts for each token, so the runtime moves only the selected experts that are not already cached to the GPU. Expert Pack reorganizes the weights of each layer/expert pair into a directly addressable contiguous expert block. The runtime reads that expert block into an aligned pinned host buffer using direct I/O, then transfers it to a GPU cache asynchronously.

This path changes how model weights are stored and delivered, not the model computation. It does not prune, replace, merge, or skip selected experts, and it does not reduce Expert Top-K. The result is a practical way to run DeepSeek-V4-Flash and the validated text-only Kimi-K3 path with one RTX 5090 (32 GB VRAM), 32 GB of CPU memory, and a 2 TB NVMe SSD.

MoE computation is sparse, but model capacity is not

Mixture-of-Experts models split the feed-forward network into many experts. After the router scores the experts for a token, only a small subset participates in that token's computation. The remaining experts are idle for that token.

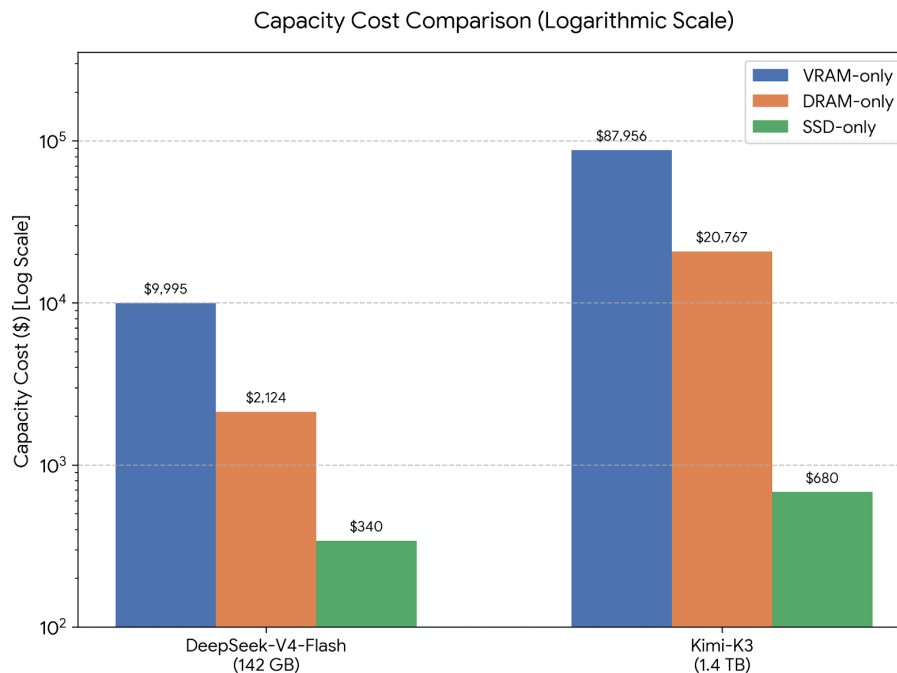
The router's choice changes across tokens and prompts. The complete expert pool must therefore remain available even though only a small working set is active at any one time. Quantization reduces the artifact size, but it does not remove the need to store the expert pool. MoE inference consequently has two different properties:

- per-token computation is sparse;
- the total expert capacity that must be stored and delivered is very large.

This is why SSD is a useful backing tier. It provides much more capacity than consumer VRAM or RAM, and modern PCIe 5.0 NVMe SSDs provide enough sequential bandwidth to make a carefully designed delivery path viable. SSD capacity becomes executable model memory only when the layout, read path, and cache policy match expert-level access patterns.

The capacity-cost difference

A capacity-cost comparison makes the trade-off clear. The following figures are capacity-only lower bounds, not complete system prices:



Capacity cost comparison for DeepSeek-V4-Flash and Kimi-K3 on a logarithmic scale

The figure does not mean SSD and DRAM have the same latency, or that buying an SSD alone is sufficient to run the model. It shows that placing the complete expert pool in VRAM or DRAM quickly becomes impractical, while using SSD for capacity and a bounded GPU cache for the active working set can substantially lower the hardware barrier.

SGLang's SSD Expert Pack approach

The central SSD-LLaMA idea is to manage SSD, RAM, and VRAM as a runtime-controlled storage hierarchy. The complete expert pool stays in the high-capacity tier, while limited VRAM retains the experts with the highest observed reuse.

SGLang's Expert Pack is an implementation of the most important expert-centric parts of that idea inside the SGLang MoE runtime:

- 1 Expert Pack makes a layer/expert pair an independently addressable contiguous expert block.
- 2 `O_DIRECT` and aligned pinned buffers remove the extra page-cache staging copy.
- 3 A byte-budgeted LFU/LRU GPU cache retains complete experts that are reused.

The current SGLang path does not claim to reproduce every mechanism in the SSD-LLaMA paper. SGLang's implementation is GPU-centric: pinned host memory is a bounded transfer staging area, not a persistent host expert cache, and the current feature does not require the paper's CPU expert execution or lossless CUDA decompression. Keeping this distinction explicit makes the feature boundary precise.

2. Why the native GGUF loading path is not enough

With the original GGUF or multi-shard tensor layout, the gate, up, and down weights of one expert may be located in different file regions. One router hit can therefore trigger several small reads, tensor-name lookups, and staging operations.

An explicit on-demand read avoids speculative prefetch, but exposes the full SSD and H2D latency on the critical path of the current MoE layer. After the router produces its result, the GPU must wait for the selected experts. Prefetching can hide part of that latency, but it has two fundamental limitations:

- the correct expert may still arrive too late because the routing result is only known after the previous computation;
- an incorrect prediction consumes SSD bandwidth, staging space, and GPU cache capacity, after which the actually selected expert must still be read.

SGLang therefore first changes the physical expert layout, then reduces the cost of every unavoidable cache miss.

Why native GGUF cannot directly use expert-level `O_DIRECT`

Native GGUF is a tensor-oriented model container, not an expert-oriented direct-I/O store. Its metadata and tensor payloads are organized around individual tensors, and a single expert's **gate**, **up**, and **down** weights may be separated across file regions or across multiple shards. The original loading path commonly uses a parser, `mmap`, or buffered file reads, so the application sees pageable page-cache-backed mappings rather than a preallocated aligned DMA destination.

`O_DIRECT` requires all of the following to be controlled by the caller:

- a file offset aligned to the storage and filesystem contract;
- a read length aligned to that contract;
- a user-provided buffer whose address is also aligned and suitable for the read.

An arbitrary tensor slice in a native GGUF file does not provide that expert-level contract. Its offset may not be aligned, its length may not be a multiple of the required block size, and the three tensors needed for one expert are not guaranteed to form one contiguous range. A caller could issue separate aligned reads with padding and then reconstruct the expert in another buffer, but that gives up the main benefit: it reintroduces multiple reads and extra assembly work, while the original `mmap`/page-cache path still cannot use the page-cache pages themselves as an `O_DIRECT` destination.

Expert Pack is the offline transformation that makes direct I/O practical. It places all roles of one layer/expert pair into one padded, aligned expert block, records its exact offset and length in the manifest, and provides a pinned buffer whose address satisfies the same contract. The runtime can then read and transfer a complete expert without asking the native GGUF layout to behave like a direct-I/O layout.

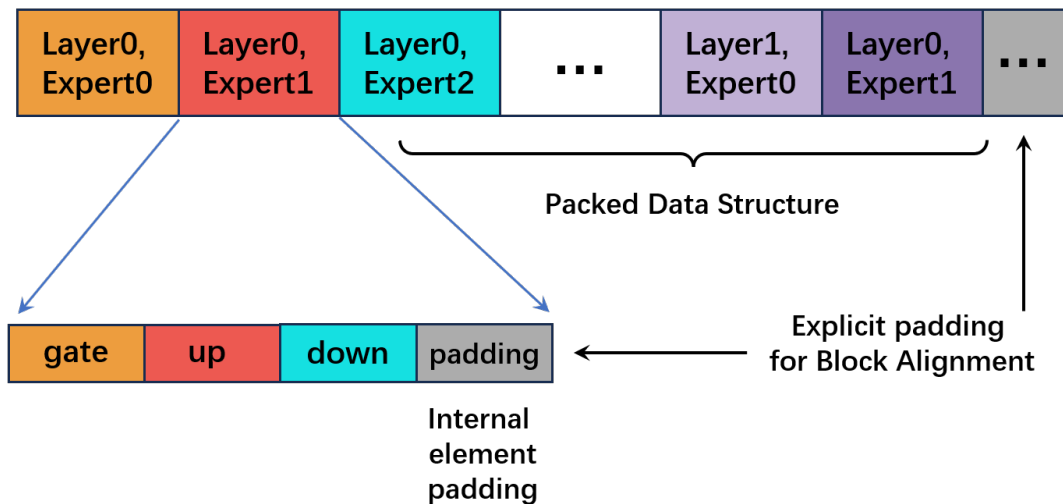
3. Expert Pack: organizing weights by expert

Contiguous expert layout

SGLang Expert Pack v1 treats one (`layer`, `expert`) pair as one complete expert. A DeepSeek expert contains the **gate**, **up**, and **down** roles. The manifest records each role's tensor boundaries, format, integrity information, and pack offset.

The logical layout is:

Expert Pack File (Physical Data Block Layout)



CORE CHARACTERISTICS: Packed Contiguous Data & Explicit Block-Aligned Padding

Expert Pack physical data block layout with contiguous expert byte-streams and explicit block-aligned padding

The runtime does not scan the file for tensor names. It derives the expert offset from the pack metadata:

```
expert_offset = data_start
               + (layer * num_experts + expert) * expert_stride
```

Role offsets inside the expert block are validated as well. One manifest lookup can therefore resolve the complete expert read range. The runtime may split that range into a bounded number of parallel tasks according to `read_splits`.

The layout changes the physical organization of weights on SSD, not their tensor contents, quantization formats, routing decisions, or model mathematics. Kimi-K3 uses a separate GGML Expert Pack adapter. The currently validated input consists of 38 Q2_K GGUF shards; its routed experts use Q2_K for gate/up and Q3_K for down.

Alignment is required for direct I/O

Direct I/O cannot use arbitrary file offsets, lengths, and user buffers in the same way as ordinary `read()`. The SGLang runtime validates:

- Expert Pack offsets for each expert;
- the start and length of every read range;
- the address of every pinned staging buffer.

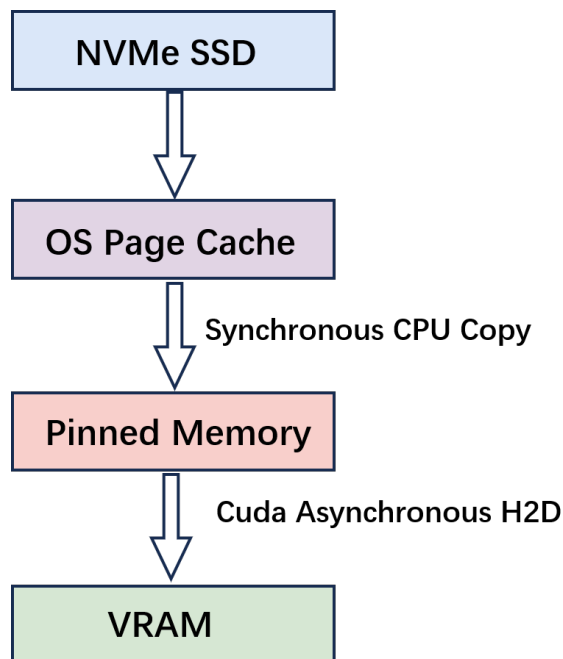
The current implementation checks 4096-byte alignment. If the pack or staging buffers do not satisfy the contract, initialization fails instead of silently falling back to an uncontrolled path during inference.

4. The key optimization: removing the page cache → pinned memory copy

This is one of the most important differences between Expert Pack and a conventional file-reading path.

Traditional buffered I/O

Traditional file reads normally go through the operating system page cache:



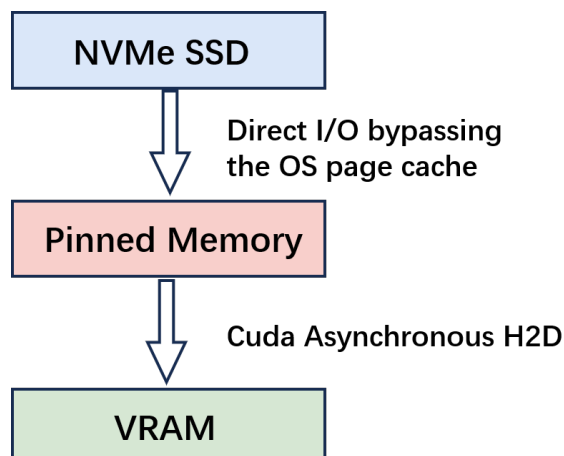
Traditional file-read path: a synchronous page-cache-to-pinned-memory copy followed by asynchronous H2D

The page cache is a kernel-managed file cache. It is not the same thing as the page-locked user memory that CUDA can use for asynchronous H2D. To issue an asynchronous H2D transfer, the application normally prepares a pinned buffer. The file data therefore has to be copied from the page cache into that pinned buffer before the GPU transfer can start. From the application's perspective, this page-cache-to-pinned handoff is a synchronous CPU memory copy: the host-side staging step must complete before the H2D operation has a valid pinned source buffer. It is not itself a `cudaMemcpyAsync` operation.

This is neither SSD reads nor H2D transfers. Rather, it is an extra synchronous memory copy operation performed by the CPU on the host side: it reads the payload from page-cache-backed memory and writes it into a pinned staging buffer. For a large expert, this translates to a read and write of the full expert size, consuming host memory bandwidth and introducing an additional kernel-to-userland staging handoff before the GPU transfer can proceed.

Expert Pack with direct I/O

When `direct_io=True`, SGLang opens the Expert Pack with `O_DIRECT` and makes the read target a preallocated, aligned pinned staging buffer:



The read target is already the pinned buffer required by CUDA, so the intermediate step below is removed:

page cache -> pinned memory

This is not a faster implementation of that copy. The copy is removed from the data path. A simplified cost model is:

```
Traditional path:
T = T(SSD -> page cache)
  + T(page cache -> pinned)
  + T(pinned -> GPU)
  + T(sync)

Expert Pack direct I/O:
T = T(SSD -> pinned)
  + T(pinned -> GPU)
```

O_DIRECT does not make the SSD's physical bandwidth increase. It removes one full host-memory traversal from the end-to-end path, which can provide these benefits:

- one less host-memory read and write, reducing CPU and memory-bandwidth pressure;
- one less synchronization handoff between kernel page cache and user-space staging;
- no large expert payload polluting the page cache and competing with unrelated data;
- a completed expert block can enter the H2D path without a page-cache staging copy;
- the same expert-level contract is reused for every layer/expert pair.

The page-cache-copy claim applies only to `direct_io=True`. The current SGLang Expert Pack loader and the DeepSeek/Kimi 5090 launchers enable this option by default. If direct I/O is explicitly disabled, the path may go through the page cache and an additional staging copy again.

5. GPU/VRAM cache: keeping the working set close to compute

The GPU cache is the mechanism that turns repeated expert access into a local VRAM hit. Expert Pack is not a cache of individual tensor fragments: one cache entry contains the complete `gate/up/down` data for one `(layer, expert)` pair. Keeping the complete expert together matters because a selected expert needs all of its roles for computation. Caching only one role would still force the other roles to be read and would not remove the cache-miss cost.

The cache is byte-budgeted rather than expert-count-budgeted. Since different models and adapters have different per-expert payload sizes, the runtime derives the number of available slots from the usable VRAM budget:

```
usable_vram = min(requested_cache, free_vram - reserve)
slot_count = floor(usable_vram / expert_payload_bytes)
```

The reserve protects memory needed by the model, CUDA runtime, activations, and other non-cache allocations. Initialization fails if the resulting slot count cannot hold one complete top-k working set. This makes the cache contract explicit: a cache budget is not allowed to consume the memory required for the current MoE computation.

On a cache hit, the runtime reuses the resident expert and does not read the Expert Pack or issue an H2D for that expert. If a previous transfer is still pending, a CUDA event protects the consumer from observing a partially installed slot. On a cache miss, the runtime selects a victim slot, reads the complete expert into a reusable pinned staging buffer, copies the expert into the GPU slot, and publishes the slot only after the transfer event is ready.

Replacement combines frequency and recency. The runtime records how often each `(layer, expert)` is selected and when it was last used. A frequently selected expert is harder to evict than a cold expert; among similarly useful entries, an older entry is a better victim. Active experts for the current top-k request are protected from eviction, so the cache cannot evict the working set it is about to execute.

The source Expert Pack is immutable. Evicting a GPU entry therefore requires no write-back: the expert can always be reconstructed from its recorded SSD offset. This makes VRAM cache management simpler than a dirty data cache and keeps replacement focused on reuse value rather than persistence.

The runtime exposes counters that make cache behavior measurable:

- `cache_hits` and `cache_misses`;
- `cache_evictions`;
- `pack_read_bytes`;
- `h2d_bytes`;
- `fallback_count` and `io_errors`.

These counters distinguish a cache problem from an I/O problem. A low hit rate means the VRAM budget or workload locality is insufficient; high `pack_read_bytes` and `h2d_bytes` with a good hit rate may instead indicate that the active set is larger than the cache during a particular phase. `fallback_count` and `io_errors` are correctness signals, not cache-performance signals.

The cost of a cache miss can be viewed as:

```
T_step ~= max(T(SSD read) + T(H2D), T(GPU compute))
```

On a GPU cache hit, both SSD read and H2D can be skipped. On a miss, the cost includes the expert read, the expert-level H2D, and the expert computation. The actual result depends on SSD bandwidth, access distribution, cache hit rate, staging-slot count, and expert shapes.

6. DeepSeek-V4-Flash and Kimi-K3 integration

6.1 Explicit opt-in loading

Expert Pack does not replace ordinary model loading. The `ExpertPackModelLoader` is selected only when the server is started with:

```
--load-format expert_pack
```

The default `auto`, `safetensors`, and `gguf` paths remain unchanged. Installing this feature therefore does not route ordinary model users into the SSD runtime by accident.

6.2 DeepSeek-V4-Flash

The currently validated input is an MXFP4 GGUF. Routed expert matrices use MXFP4. Non-routed weights are provided by the GGUF iterator, while the Expert Pack runtime handles dynamic routed-expert delivery.

With the GGUF and runtime environment prepared, use the 5090 launcher:

```
python3 -m pip install -e 'python'

python3 examples/runtime/deepseek_v4/benchmark_deepseek_5090.py \
  --gguf /path/to/deepseek-v4-flash-0731-mx4p4.gguf \
  --prompt 'Explain why the sky appears blue.' \
  --max-new-tokens 200
```

The launcher uses `--load-format expert_pack`, prepares or validates model metadata, Expert Pack, and manifest artifacts, and passes runtime settings through `--model-loader-extra-config`. The first launch prepares the pack; later launches reuse the existing artifacts.

6.3 Kimi-K3

The currently validated input is the text-only 38-shard GGUF path, not the complete multimodal safetensors checkpoint. Point the launcher at the first shard; it discovers the remaining shards in the same directory:

```
python3 examples/runtime/kimi_k3/benchmark_kimi_k3_5090.py \
  --gguf /path/to/kimi-k3/KIMI-K3-MXP4-DERISKED-Q2_K-00001-of-00038.gguf \
  --prompt 'Explain why the sky appears blue.' \
  --max-new-tokens 200
```

Kimi uses the GGML Expert Pack adapter. The validated routed experts use Q2_K for gate/up and Q3_K for down. Other GGUF quantization variants should not be claimed as supported without additional validation.

6.4 Model and Expert Pack weight footprint

The validated original model weights and generated Expert Pack files occupy:

Model	Original weight files	Weight size	Expert Pack size
DeepSeek-V4-Flash	One Ollama MXP4 GGUF blob	155.10 GB	147.18 GB
Kimi-K3	38 Q2_K GGUF shards	1009.51 GB (about 1.01 TB)	985.61 GB

These are file sizes for the validated weight payloads and generated Expert Pack payloads. Expert Pack indexes, locks, and other metadata are separate and are not included in this table.

7. Correctness boundary

Expert Pack is a weight-layout and delivery optimization, not an approximate-inference algorithm. The correctness contract is:

- every expert selected by the router is executed;
- Expert Top-K is unchanged;
- a selected expert is not replaced by a different resident expert;
- selected experts are not pruned, skipped, or merged;
- the pack and manifest are structurally, dimensionally, and cryptographically validated as configured;
- `fallback_count` and `io_errors` are reported instead of silently hiding I/O failures.

Validation showed that DeepSeek-V4-Flash produced semantically equivalent answers to Ollama across multiple prompt categories. Kimi-K3 matched the 200-token SGLang reference output; all 92 routed layers executed Top-16 experts with `fallback_count=0` and `io_errors=0`.

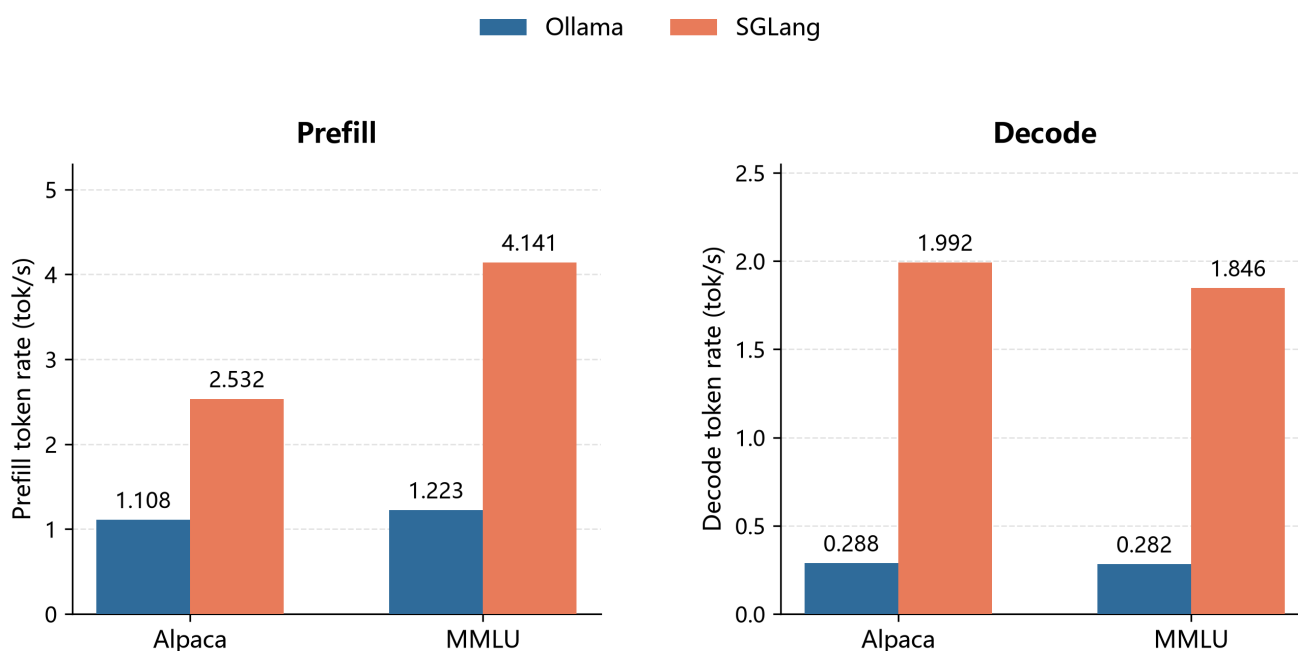
8. Performance results

All SGLang, Ollama, and llama.cpp measurements use the same test environment: one NVIDIA RTX 5090 with 32 GB VRAM, 32 GB of CPU memory, and a 2 TB NVMe SSD. The figures below describe validation results under this shared hardware condition; token counts and runtime-specific software settings remain as stated in each comparison. The earlier summary tables are retired; the figures below are now the canonical presentation of the token-rate comparison.

DeepSeek-V4-Flash vs. Ollama

The comparison uses ten shared requests: five Alpaca and five MMLU. Both runtimes generated up to 200 tokens per request. The chart reports mean prefill and decode token rates for each dataset.

DeepSeek-V4-Flash: SGLang vs Ollama token rates



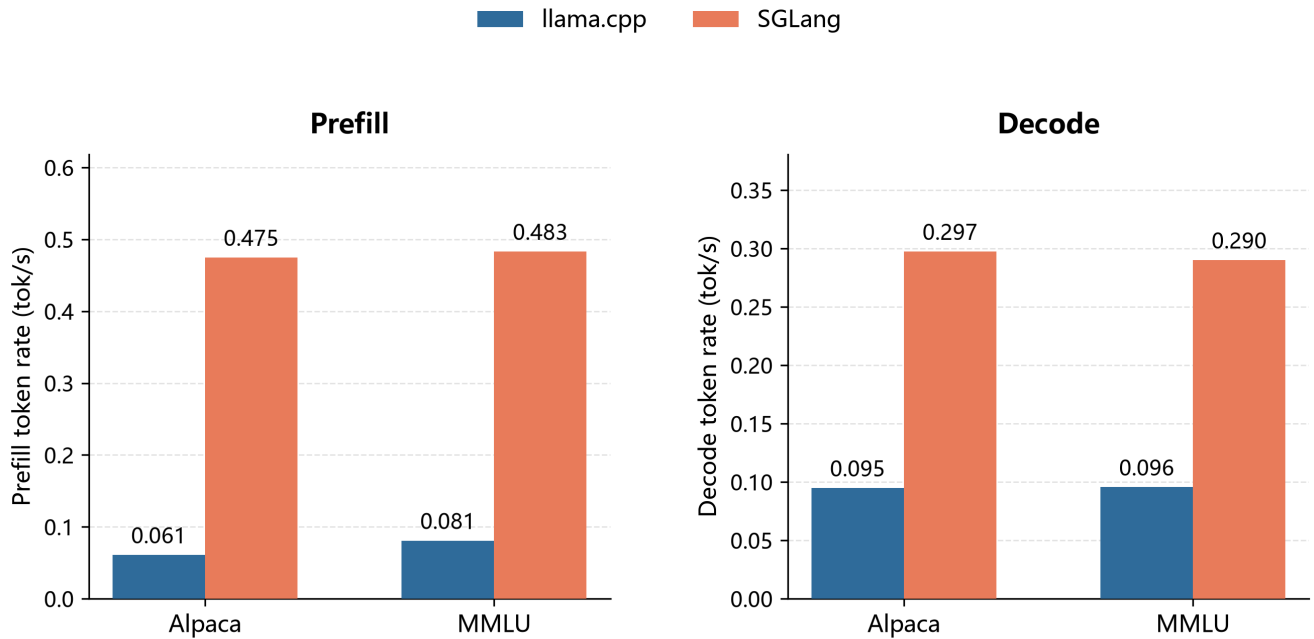
DeepSeek-V4-Flash SGLang versus Ollama prefill and decode token rates for Alpaca and MMLU

Relative to Ollama, SGLang improves prefill by 2.28x on Alpaca and 3.39x on MMLU. Decode improves by 6.92x and 6.55x, respectively.

Kimi-K3 vs. llama.cpp

The comparison includes three Alpaca and two MMLU samples. SGLang generated 200 tokens, while llama.cpp generated 50 tokens, so the decode comparison is indicative rather than a strict equal-length A/B measurement. The chart reports mean prefill and decode token rates for each dataset.

Kimi-K3: SGLang vs llama.cpp token rates



Kimi-K3 SGLang versus llama.cpp prefill and decode token rates for Alpaca and MMLU

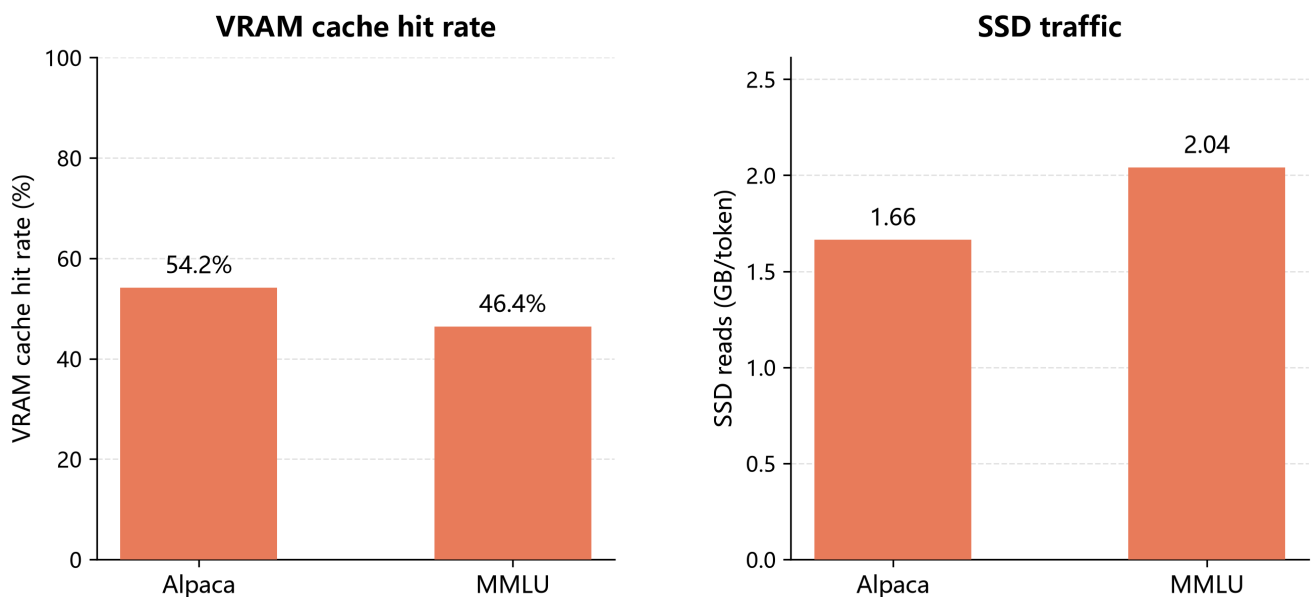
Relative to llama.cpp, SGLang improves prefill by 7.81x on Alpaca and 5.98x on MMLU. Decode improves by 3.14x and 3.04x, respectively.

Expert-cache hit rate and SSD traffic

Token rate should be read together with cache telemetry. The following Python-generated bar charts report two independent quantities: the share of routed-expert accesses served from the VRAM cache, and decimal gigabytes read from the Expert Pack per generated token. A cache hit avoids the SSD read and H2D transfer for that expert; GB/token also depends on expert payload size and routing locality.

Both figures show SGLang-only telemetry, so the single series is labeled by the surrounding text rather than a legend.

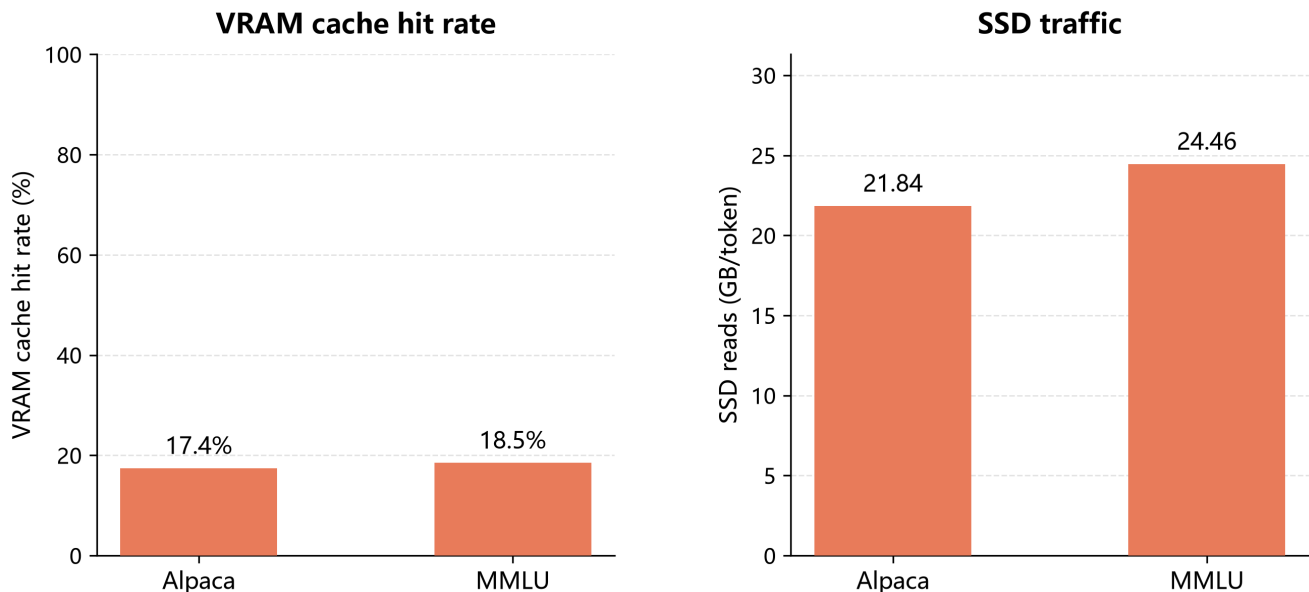
DeepSeek-V4-Flash: expert cache and SSD traffic



DeepSeek-V4-Flash SGLang VRAM cache hit rate and SSD reads per generated token

DeepSeek uses the complete ten-request run: five Alpaca and five MMLU requests, each with a 200-token completion. Its hit rate is 54.2% for Alpaca and 46.4% for MMLU, with 1.66 and 2.04 GB read per generated token.

Kimi-K3: expert cache and SSD traffic



Kimi-K3 SGLang VRAM cache hit rate and SSD reads per generated token

The Kimi figure uses the first three completed requests of the interleaved run: two Alpaca and one MMLU, each with a 200-token completion. The current means are 17.4% and 18.5% VRAM cache hit rate, with 21.84 and 24.46 GB read per generated token, respectively. The remaining seven requests are still running, so these are explicitly stage results rather than final ten-request averages. The difference from DeepSeek is expected: Kimi's configuration reserves 5 GiB for the GPU expert cache, while the DeepSeek run reserves about 21 GiB, and the two adapters have different expert payload sizes and routing behavior.

The improvement does not come from one isolated faster-copy primitive. It is the combined effect of:

- 1 Expert Pack turning scattered tensor accesses into addressable contiguous expert reads;
- 2 direct I/O removing the page cache -> pinned memory copy;
- 3 pinned staging provides a CUDA-compatible host source for expert-level H2D without page-cache staging;
- 4 the GPU cache skipping SSD reads and H2D transfers on a hit.

9. Conditions and limitations

SSD capacity and preparation time

Expert Pack requires additional SSD capacity. The PR records an estimated 5-10 minutes to build the DeepSeek-V4-Flash pack and approximately 8-15 minutes for first-run readiness. Kimi-K3 pack construction took 29 minutes 42 seconds in the retained measurement, with approximately 35-45 minutes to first readiness. The 38 Kimi source shards and generated Expert Pack occupy about 1.814 TiB in total; the measurements use a 2 TB SSD, while a 4 TB SSD is recommended for practical deployment headroom.

Direct I/O

O_DIRECT requires platform support and alignment of file offsets, read lengths, and user-buffer addresses. SGLang fails closed during runtime initialization. If the platform does not support direct I/O, or the pack does not satisfy the alignment contract, the result should not be described as direct-I/O performance.

Cache and workload

- A smaller GPU cache creates more misses, putting SSD reads and H2D transfers on the critical path more often.
- A change in prompt or workload distribution can change the hot experts, so one request's hot set is not guaranteed to fit every workload.
- If the complete expert pool already fits in GPU memory, Expert Pack adds an unnecessary data path and is not the right deployment mode.
- If the workload is dominated by GPU computation, the benefit of removing the host copy may be hidden by compute time.
- If SSD random-read behavior, queueing, or thermal stability is poor, increasing `read_splits` and staging slots may add queueing and memory pressure instead of throughput.

Current feature boundary

This feature focuses on routed-expert SSD delivery and GPU caching. It does not provide SSD KV-cache offload and does not change request scheduling. It is an explicit opt-in Expert Pack path, not a global replacement for every SGLang model-loading format.

10. Conclusion

SSD Expert Pack is not simply replacing GPU memory with a slower disk. It redesigns weight delivery around the sparse access pattern of MoE inference:

```
router selects a small set of experts
-> Expert Pack resolves their offsets
-> O_DIRECT reads into aligned pinned buffers
-> expert-level asynchronous H2D fills the GPU cache
-> the complete expert becomes available for computation
```

Removing the `page cache` -> `pinned memory` copy is an easy detail to miss, but it is a concrete end-to-end optimization. A traditional path reads into the OS page cache and then copies the payload into CUDA-usable pinned memory. With direct I/O, Expert Pack uses the preallocated pinned buffer as the read target and removes that full host-memory movement and synchronization handoff.

This is how SGLang applies the core SSD-LLaMA idea to real DeepSeek-V4-Flash and Kimi-K3 integrations: the complete expert pool remains on high-capacity SSD, a bounded GPU cache retains the current working set, and the runtime moves only the experts selected by the router. Very large MoE models no longer require stacking enough VRAM or DRAM to hold the complete model and can instead run on a consumer GPU paired with a high-speed NVMe SSD.